Team Project Report - Phase 1

Phase 1 summary

Team name: dwendwen

Members (names and Andrew IDs):

Qing Deng (qingd)

Jiarui Zhang (jiaruiz)

Wangshu Li (wangshul)

Performance data and configurations

Database choosed (MySQL/HBase): MySQL

Number and types of instances: 7 m6a.large instances

Cost per hour of the entire system:

EC2: 0.0864 * 7 = 0.6048

EBS: (32 + 20*2 + 64*6 + 60) * 0.1/720 = 0.0716

ELB: 0.0225

Total: $0.6989 / hour

(Cost includes on-demand EC2, EBS and ELB costs. Ignore S3, EMR software addition, Network and Disk I/O costs)

Requests Per Second (RPS) of your <u>best</u> submission:

	M1	M2	M3
score	8.20	50.00	
submission id	696535	696454	
throughput	53302.13	41001.58	
latency	8.16	9.65	
correctness	100.00	88.35	
error	0.00	0.00	

Rubric

- Each unanswered bullet point = -4%
- Each unsatisfactory answer = -2%
- Use the report as a record of your progress, and then condense it before submitting it.
- If you write down an observation, we also expect you to try and explain it.

- Questions ending with “Why?” need evidence (not just logic).
- Always use your own words (paraphrase); we will check for plagiarism.

Task 1: Web-tier

Question 1: Comparing web frameworks

Microservice 1 and 2 does not involve database access and is a good opportunity for you to compare and choose a suitable web framework for future queries. Please try a couple of combinations of programming and web frameworks, and answer the questions below. To save cost during your comparison, we recommend that you deploy your testing code to a single EC2 virtual machine instead of a Kubernetes cluster.

- You are free to pick either M1 and M2 (or both) to compare web frameworks. Which Microservice did you choose and why?
    - We chose M1 because we wanted to select a suitable framework as early as possible.
- What frameworks have you tried? How did each framework perform? Please include instance (or cluster) configuration, latency and RPS of each framework you tried.

framework	instance/cluster configuration	latency	RPS
Python - Django	m6a.large/(6 worker nodes, 1 master node, 30 replica, ALB)	178.18	2493.65
Java - Vert.x	m6a.large/(6 worker nodes, 1 master node, 30 replica, NLB)	8.20	53302.13
Java - Jooby	m6a.large(6 worker nodes, 1 master node, 30 replica, NLB)	6.27	40784.94
Java - SpringBoot	m6a.large(6 worker nodes, 1 master node, 24 replica, ALB)	3.27	21223.30

- For the two frameworks with the highest RPS, please list and compare their functionality, configuration, and deployment.

	vert.x	jooby
functionality	We used Vert. x-Web to build our web application backend server. The major features of it include asynchronous	Jooby is a scalable and fast micro web framework that could achieve high performance and high parallelization.

configuration	Add vertx dependency at maven and several Verticle.java files.	Add jooby dependency at maven; only one App.java file
deployment	Build the docker image and deploy to the Kubernetes cluster	Build the docker image and deploy to the Kubernetes cluster

- Which one would you like to choose for other queries? What are the most important differentiating factors?

  Vert.x is the selected Java framework for the reported AWS runs, with Jooby retained as another implementation. The measured RPS, deployment effort, documentation and MySQL support guide that choice. The web tier also includes C# 12 / .NET 8 projects in web/m1_dotnet, web/m2_dotnet and web/m3_dotnet. ASP.NET Core Minimal APIs expose the same three routes. m1 implements the custom QR format; m2 validates chains and mines blocks; m3 retrieves and ranks contacts from MySQL. Each project has its own entry point, service classes and configuration, so containers can be built and scaled independently. MySqlConnector provides asynchronous database access for m3.

- Did you have to do any trade-offs while picking one of the multiple frameworks you selected?

  The main trade-off is development convenience versus throughput and control over concurrency. Django supports quick prototyping, while the Java measurements favor Vert.x and Jooby over Spring Boot for this workload. ASP.NET Core provides compact route definitions and built-in JSON, compression and cryptographic APIs. Its m1 codec retains the custom 21 x 21 / 25 x 25 matrices, Logistic Map XOR and 32 x 32 decoding canvas. m2 uses zlib, URL-safe Base64, SHA-256 and BigInteger accounting, checks hashes and signatures, and supports cancellation during mining. These CPU-oriented handlers benefit from suitable resource allocation. m3 instead uses asynchronous SQL and a separate pooled connection per request; shared services do not hold mutable request state.

Question 2: Load balancing and scaling

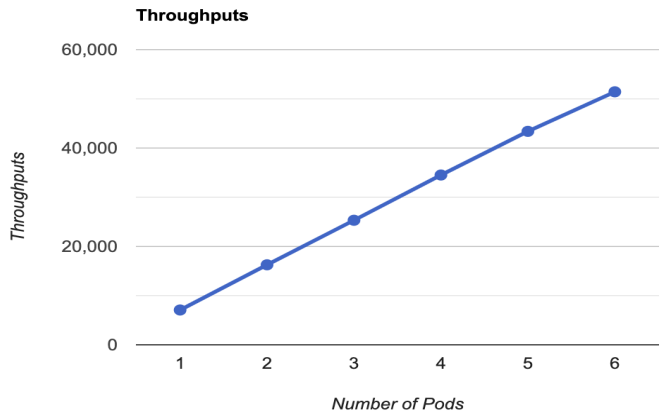
Our target RPS for Microservice 1 and 2 are 65k and 25k respectively, which can be hard to reach with a single pod. We have learned about Kubernetes and ELBs in the individual projects, so let us explore the knowledge and skills you have developed here.

- Discuss load-balancing in general, why it matters in the cloud, and how it is applicable to your web-tier.

Load balancing lets you evenly distribute network traffic to prevent failure caused by overloading a particular resource. This strategy improves the performance and availability of applications, websites, databases, and other computing resources. It also helps process user requests quickly and accurately.

In our web-tier, load balancing distributed workload to different pods in different worker nodes, which prevents congestion and improves performance.

- Provision a cluster with only one worker node and deploy your Microservice 1 using a Kubernetes deployment with one replica. Provision a Network Load Balancer by using Kubernetes service resource. Make a 120-second submission to the Sail() Platform and record the RPS. Add one replica to your deployment at a time, while keeping your worker node count to only 1. Wait until the target group shows the new pod as healthy, make another submission to the Sail() Platform and record the RPS. Repeat this process for 2 times, ending with 3 replicas running on 1 worker node.
  As the number of replicas increases, does the RPS grow accordingly? Please show the graph with the data points you just collected and write down a possible explanation.
  One replica: 7159.94
  Two replicas: 8879.16
  Three replicas: 8819.66
  The throughput increases a little from one replica to two replicas but does not increase from two replicas to three replicas. The reason may be that one replica didn't utilize the full CPU but two replicas utilized the full CPU. Since two replicas already utilized the full CPU, increasing replicas to three would not increase the throughput.
- Provision a cluster with as many worker nodes as possible while staying under the $0.70/h submission budget. Deploy your Microservice 1 using a Kubernetes deployment with only one replica and provision a Network Load Balancer by using Kubernetes service resource. Make a 120-second submission to the Sail() Platform and record the RPS. Add one replica to your deployment at a time, wait until the target group shows the new replica as healthy, make another submission to the Sail() Platform and record the RPS. Repeat this process until your number of replicas reaches your number of the worker nodes.
  As the number of replicas increases, does the RPS grow accordingly? Please show the graph with the data points you just collected and write down a possible explanation.



Yes, the RPS grows accordingly, because each new replica is on a new instance and utilizes the new CPU. Therefore, the overall CPU utilization increases accordingly.

- How do you check which pod is running on which worker node? Give a specific command. How would you ensure that the workload is distributed evenly across your cluster?
  kubectl get pods -o wide
  After running this command, we can see which worker node each pod is. If we see a roughly equal number of pods in each node, then the workload is evenly distributed.
- Compare the difference between Application Load Balancer and Network Load Balancer. For each type, use the best cluster configuration you can have under the $0.70/h submission budget. Use the ingress resource to provision an ALB and service resource for NLB. Wait until the load balancers are in active state and the target groups show the pods as healthy. Submit the ALB/NLB endpoint to the Sail() Platform (M1, 600s) 6 times consecutively and record the RPS. Finish the following table.
  Note: Each type requires 6 submissions * 10 minutes consecutively, so it will take at least 1 hour, please plan your time accordingly.

  The instance type of your master / worker nodes: m6a.large
  The number of instances in your cluster: 1 master, 6 worker

	Application Load Balancer	Network Load Balancer
Submission 1	22468.87	46728.98
Submission 2	34241.61	50272.53
Submission 3	43789.83	52384.09
Submission 4	46982.25	51234.83
Submission 5	48392.16	52334.15
Submission 6	47190.86	52124.65

- We mentioned ALB warm-up in individual project P1. Explain the warm-up and why it is necessary based on the experiment above.
  Application Load Balancer is designed to handle large loads of network traffic when the traffic increases gradually. However, if the traffic load increases rapidly over a short period of time, it becomes difficult for the ALB to handle this load fast enough. In this case, warm-up can make the clusters scale up before the actual load spikes.
  Based on the experiment above, we can see that the second submission of ALB increased throughput by over 10000 compared to the first submission. This shows that warm-up is indeed necessary.
- We discussed scaling Kubernetes deployment in P2. Besides manually specifying the number of replicas in your deployment resource, describe one other technique that scales the web-tier of your microservice.

Maybe we can use autoscaling. For example, create a cloud watch to measure the CPU utilization and scale out or scale in according to the specific rules.

Question 3: Web-tier architecture

This question applies to all microservices. For each microservice we need a web-tier that interprets requests, fetches data, optionally retrieves data from the database and does processing to generate responses. How would you set up your web-tier for these purposes? You can use the following questions as a guide.

Keep in mind that, although you can test different microservices individually for now, in later phases your 3 microservices will need to be under the same public DNS and respond to requests at the same time. Make sure to consider this in your architecture and cluster design.

- Which web-tier architecture did you choose? Did you put pods of different microservices into the same worker node? If you choose MySQL as your storage-tier, did you put the pods of your web-tier of microservice 3 and MySQL pods into the same deployment?

  Ingress routes requests to separate service Deployments, including ASP.NET Core containers. Web pods can share worker nodes; MySQL remains a separate deployment.
- What alternative architectures can you think of? What are their implications on performance and scalability and why?

  A combined Java application can route all requests in one process, but releases and scaling then affect the whole application. Separate Java or C# service containers support independent deployment and replica counts. On AKS, an NGINX gateway exposes one public LoadBalancer and routes /qrcode, /blockchain and /twitter to internal Services. This keeps the public API consistent while each service has its own lifecycle.
- Explain your choice of instance type and the number of worker nodes for your Kubernetes cluster.

  We chose m6a.large because it has a good performance/cost ratio. The number of worker nodes is 6, because this is the maximum number within the budget.
- (Optional) Have you tried more than one architecture or cluster configuration? What did their performance look like? Can you reason about the difference?

Question 4: Web-tier deployment orchestration

We know it takes dozens (or hundreds) of iterations to build a satisfactory system, and the deployment process takes a long time to complete. Setting servers/clusters up automatically saves developers from repetitive manual labor and reduces errors.

- We require you to use Terraform, Kubernetes and Helm to orchestrate the deployment of your web-tier. What steps were taken every time you deployed your system? Please include your Terraform script, Helm chart and Kubernetes manifest in your submission as per the requirements stated in the Phase 1 writeup. If you use kOps to provision your Kubernetes cluster, please also include the cluster definition YAML files in your submission. Starting from a clean local machine (with kops, kubectl and helm installed),

we should end up with a Kubernetes cluster which has a functioning web-tier by applying your deployment steps.

For AWS, create the S3 cluster-state bucket and set KOPS_STATE_STORE, then apply cluster.yaml, ig-master.yaml and ig-nodes.yaml with kOps. Run kops update and kops validate, and install the m1, m2, m3 and MySQL Helm charts under k8s/helm. Remove the cluster with kops delete when it is no longer required.

For Azure, phase3-deployment/AzureDevOps contains an infrastructure pipeline for the resource group, ACR and AKS, plus an application pipeline. Select dotnet, vertx or jooby and the dev/prod environment. Validation checks Shell syntax and renders the Helm chart; build jobs package separate service images. Main/master builds push images tagged with the Build ID to ACR.

The deployment job authenticates to AKS, creates a build-specific MySQL Secret and runs helm upgrade --install --atomic --wait. The chart creates application Deployments, internal Services and the NGINX gateway. m3 connects to a reachable MySQL database populated by ETL. Route checks follow the rollout; rollback.sh selects an earlier Helm revision when needed. Cloud access uses a service connection, and database credentials come from secret variables.

- If Terraform, Kubernetes and Helm are not required, what are some other ways to automate your web-tier deployment? Compare their advantages and disadvantages.
  There are some other development automation tools, such as Rancher and Ansible which can automate the deployment. Take the comparison between Terraform and Ansible as an example.

	Terraform	Ansible
Advantages	best fit for orchestrating cloud services and setup cloud infrastructure from scratch.	provides full support for packaging and templating; supports the provisioning of bare metal servers
Disadvantages	does not provide better support in terms of packaging and templating.	does not have lifecycle management; mainly used for configuring servers with the right software and updates already configured resources

- How can you further automate the entire deployment process? If you implemented any other automation tools please describe them, and we would like to see your scripts in your code submission.

Azure DevOps links validation, builds and releases to repository changes. Pull requests validate and build; main/master publishes images and deploys the selected environment. Separate dev/prod configuration controls cluster targets and replica counts, and Build ID tags connect each Helm release to its images.

Task 2: Extract Transform Load (ETL)

Question 5: Your ETL process

We have 1TB of raw data, which takes a non-trivial amount of time and money to process. It is strongly advised to test run and debug your E & T on a small portion of the data locally, and plan wisely before booting expensive clusters! Please briefly explain:

- The programming model/framework used for the ETL job and justification.
    - For ETL jobs, we used Spark written in Scala to do the ETL job. We did some research about PySpark as well. However, since we are exposed to Scala in P4 and Spark is written in scala, we choose to write in Scala rather than PySpark.
- The number and type of instances used during ETL and justification.
    - We used 1 master node and 4 worker nodes of type n1-standard-4(4 vCPU and 15 GB memory), with 899 GB primary disk size each. Given the memroy and CPU quota, this is the best configuration, in terms of CPU and memory, we could do.
- The execution time for the ETL process, excluding database load time.
    - For filtering malformed tweets, extracting needed columns, it took roughly 3 minutes to perform all the transformations.
    - We also create three other tables, whose transformations cost about 5 minutes in total.
- The time spent loading your data into the database.
    - For table *tweets*(which contains the filtered dataset and needed columns), it took around 3.5 hours to load the dataframe into MySQL database.
    - For table *users*, *hashtags* and *scores*, they cost roughly 1.5 hour each.
- The overall cost of the ETL process.
    - $89
- The number of incomplete ETL runs before your final run.
    - 3
- The size of the resulting database and reasoning.
    - 18.152 GB. The size is way smaller than 1T because we eliminate many unnecessary columns. We only keep the columns that help us to calculate the three scores as well as output user information.
- The size and format of the backup.
    - 23GB, sql file format.
- Discuss the difficulties encountered.
    - **Backup took so long to load into MySQL server on EC2:** We decided to host MySQL server on EC2 during development process due to budget consideration. However, simply loading a 23GB backup with mysql command would cost around 3

days. Thus, we searched online for possible solutions to accelerate the loading process.
    - Not sure how to calculate Hashtag scores during ETL. So, we temporarily put hashtags of one user into a separate table and do the logic in Java code.
- If you had multiple database schema iterations, did you have to rerun your ETL between schema iterations?
    - Yes. Since some of the tables depend on some other tables. We have to rerun ETL between schema iterations.

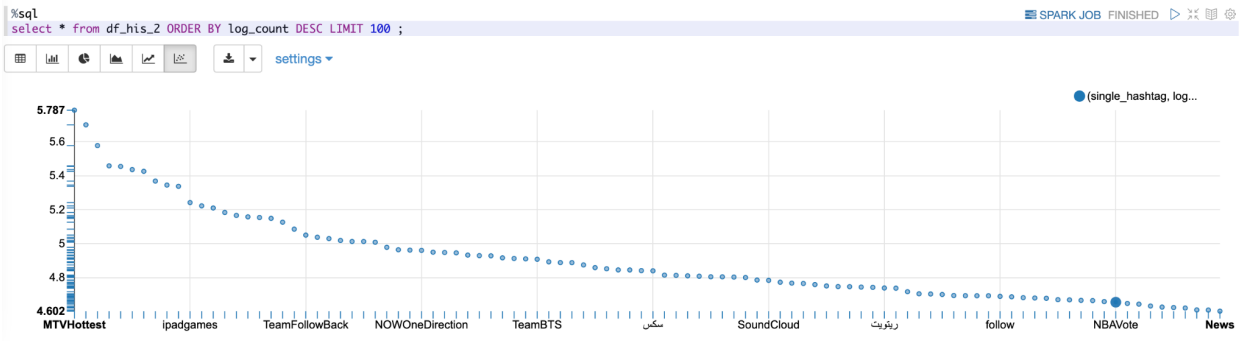
Question 6: ETL considerations

Historically, some teams ran out of the budget because they chose AWS for ETL and had multiple runs before they got their final dataset. Sometimes they simply used unnecessarily fancy (expensive) hardware. We believe you have compared and contrasted all the options and made smart decisions. Here are a few more questions about your understanding of the different choices:

- Which cloud service provider (AWS, GCP, Azure) and which service (EMR, Dataproc, HDInsight, Databricks, etc.) did you use and what was the difference?
  We choose GCP with a Dataproc cluster for the ETL process. I think the difference between Dataproc and HDInsight is that Dataproc is easier to deploy and the cost is lower. Besides, the Zeppelin notebook in Dataproc supports .zpln and .ipynb files but HDInsight only supports .json files. In general, HDInsight and Dataproc are both very good for Apache Spark management.
- What are the most effective ways to speed up ETL?
  The most effective ways to speed up ETL are:
    1. Deploy more worker nodes and upgrade the master node as well as worker nodes.
    2. During the test period, we can use as many transformations and actions(like df.show() or rdd.collect()) as we want to ensure the correctness. Once we are confident about the confidence, we can remove all intermediate actions and leave just 1 final action which is loading the dataframe to the database.
- Besides Spark, list two other programming models you can use for ETL. Which approach would be more efficient and why? The term efficient can refer to many different aspects such as development time and actual run time, but as a result high efficiency should reduce the final ETL cost.
  We mainly use Spark for ETL. We can also use pandas in Python or Perl for ETL. Spark would be more efficient because Spark is a distributed programming model designed for large-scale data processing. By using spark. The large-scale raw data can be processed in multiple worker nodes concurrently which save lots of time compared to standalone programming models. The development time for Dataproc cluster is trivial compared to run time. So Spark is definitely more efficient.
- Did you encounter any error when running the ETL job in the provisioned cluster? If so, what are those errors? What did you do to investigate and recover from the error?
  The major error we encountered during ETL is the session between localhost and the master node dies and the code stops running. We investigated the code carefully and found that we used too many rdd.collect() actions when mapping rdds. As mentioned in the

primer, the collect() action will push the data from worker nodes to the master node, which will consistently consume the memory of the master node. When the master node runs out of memory, the session will die and the code will stop running. We just switched into another approach to map the rdd to get rid of the collect() action and finally recover from the error.

- How did you load data into your database? Are there other ways to do this, and what are the advantages of each method? Did you try some other ways to facilitate loading?
  We follow the sample ETL code in the primer, which calls dataframe.write.jdbc() to load the data into our database. Another way to do this is save the dataframe as a csv file and store it inside the cloud storage bucket(we store it in gcp storage bucket), then we use sql command to load the csv file into the database. The advantage of loading the dataframe directly into the database is that it does not need an intermediate csv file and it can also handle special characters(like '\n' and '\r\n') in text correctly. The loading time is relatively trivial so we didn't use other ways to facilitate loading.
- Did you store any intermediate ETL results and why? If so, what data did you store and where did you store the data?
  As mentioned in the previous answer. We first tried to store the ETL results into a csv file and store it in the gcp storage bucket and then use sql command to load the csv file into the database. But there exists a potential problem in this intermediate csv file, which is the special characters like '\n' and '\r\n'. So finally we choose to directly load the dataframe into the database.
- We would like you to produce a histogram of hashtag frequency on **log** scale among all the valid tweets **without hashtag filtering**. Given this histogram, describe why we asked you to filter out the top hashtags when calculating the hashtag score. [Clarification: We simply need you to plot the hashtag frequencies, sorted on the x-axis by the frequency value on the y-axis]



Based on the histogram, we could observe that these popular hashtags could contribute a large portion of the hashtag score, and make the absolute value really big, which might diminish the contribution of the other two scores, interaction score and keyword score, since there absolute value is way smaller than hashtag score. Thus, we should remove the popular tags to normalize the result.

- Before you run your Spark tasks in a cluster, it would be a good idea to apply some tests. You may choose to test the filter rules, or even test the entire process against a small dataset. You

may choose to test against the mini dataset, or design your own dataset that contains interesting edge cases. **Please include a screenshot of the ETL test coverage report here.** (We don't have a strict coverage percentage you need to reach, but we do want to see your efforts in ETL testing.)

- Note: Don't forget to put your code for ETL testing under the **etl** folder in your team's github repo **master** branch.

We designed our own mini dataset(around 10 rows) to test the correctness of our code. Specifically, we added some edge cases such as user1 and user2 that have multiple contacts including replies and retweets. We ran our code on the mini dataset and then checked the result to make sure that it's the same as the result we manually computed. Here are some screenshots of the ETL testing results

```
val interaction_agg_pair_join = spark.sql(
    """
    SELECT _1 AS user_pair, SUM(_2) AS reply_count, SUM(_3) AS retweet_count
    FROM interaction_agg_pair
    GROUP BY user_pair
    ORDER BY reply_count+retweet_count DESC
    """)
interaction_agg_pair_join.show()
```

```
+---------+-----------+-------------+
|user_pair|reply_count|retweet_count|
+---------+-----------+-------------+
|    12 14|          1|            1|
|    11 13|          2|            0|
|    12 13|          0|            1|
|    13 14|          1|            0|
|    11 11|          0|            1|
|    11 14|          1|            0|
|    11 12|          0|            1|
+---------+-----------+-------------+

interaction_agg_pair_join: org.apache.spark.sql.DataFrame = [user_pair: string, reply_count: bigint ... 1 more field]
```

```
val final_result = spark.sql(
    """
    SELECT interaction._1 AS user1, interaction._2 AS user2, interaction._3 AS interaction_score, hashtag.score AS hashtag_score
    FROM interaction_result AS interaction INNER JOIN hashtag_result AS hashtag
    ON interaction._1 = hashtag.small AND interaction._2 = hashtag.big
    """)
final_result.show()
```

```
+-----+-----+------------------+-------------+
|user1|user2| interaction_score|hashtag_score|
+-----+-----+------------------+-------------+
|   12|   13|0.6931471805599453|          1.0|
|   13|   14|1.0986122886681098|          1.0|
|   11|   14|1.0986122886681098|          1.0|
|   11|   12|0.6931471805599453|          1.0|
|   11|   11|0.6931471805599453|          1.0|
+-----+-----+------------------+-------------+

final_result: org.apache.spark.sql.DataFrame = [user1: bigint, user2: bigint ... 2 more fields]
```

```
+---------+-------------+---------+--------------------------------+
|user_id_a|all_hashtags_a|user_id_b|all_hashtags_b                  |
+---------+-------------+---------+--------------------------------+
|861      |[CBInsights] |861      |[CBInsights]                    |
|861      |[CBInsights] |874      |[WHDemoDay]                     |
|861      |[CBInsights] |5556     |[AKinAmethi, OurHopeAAP]        |
|861      |[CBInsights] |5622     |[PackageDesignDrinkingGame]     |
|861      |[CBInsights] |10399    |[Athnet]                        |
|861      |[CBInsights] |11569    |[JetsfollowJets]                |
|861      |[CBInsights] |11830    |[NHS, coding]                   |
|861      |[CBInsights] |26253    |[rtfm]                          |
|861      |[CBInsights] |491123   |[hull]                          |
|861      |[CBInsights] |625303   |[LFC]                           |
|861      |[CBInsights] |633443   |[BalloonRelease]                |
|861      |[CBInsights] |761906   |[24A, DaleRT]                   |
|861      |[CBInsights] |768058   |[Jacobin, Merry4Nivôse]         |
|861      |[CBInsights] |771698   |[e20s]                          |
|861      |[CBInsights] |800835   |[RIPCarrieFisher]               |
|861      |[CBInsights] |862771   |[DDoS, defense, deloittepredicts]|
|861      |[CBInsights] |1123601  |[citizensassembly, repealthe8th]|
|861      |[CBInsights] |1434881  |[JennyHan, LaraJean]            |
|861      |[CBInsights] |1448211  |[mPLUSRewards, Nexercise]       |
|861      |[CBInsights] |1569091  |[SPN, supernatural, lovetheseboys]|
+---------+-------------+---------+--------------------------------+
```

Task 3: Databases (MySQL or HBase)

Question 7: Database system

For Microservice 3, you are free to choose between MySQL or HBase. In the checkpoint report, we asked you to explore the differences between MySQL and HBase.

- Which database system did your team choose eventually, and why?
  We finally choose to use MySQL. There are several reasons for our choice. First, we need 3 tables for the web tiers, which are the interaction score table, hashtag score table, and the filtered data table. Although the raw data has a very large volume(1 TB). But the filtered data table, interaction score table and hashtag score table are relatively small. By setting up the index on each table, we can quickly query for the information we need in the web tier. Second, Hbase requires careful design of the row key. During the developing and testing period, the schemas of our database tables are often changing, which results in refactoring of row key design. It's very time-consuming so we don't choose Hbase.

Question 8: Schema

We first extract and transform the raw data and then store it in databases in a good format. Various schemas can result in a very large difference in performance! In Phase 1 we only ask you to reach 6% of the target RPS (600 out of 10000) of Microservice 3 for receiving checkpoint score. If you achieve 2000 rps, you will get the M3 Early Bird Bonus. These targets can be achieved with a basic but functional schema. In Phase 2 your Microservice 3 will need to reach 100% of the target (10,000) RPS which would require a more efficient schema.

When you are optimizing your schema in Phase 2, you will want to look at different metrics to understand the potential bottlenecks and come up with a better solution (before trying to tune parameters!). These iterations take a lot of time and a lot of experimentation so keep this in mind while designing your schema in Phase 1.

- What is your schema for Microservice 3 in Phase 1? Did you already have multiple iterations of schemas? If so please describe the previous schemas you have designed and explain why you decide to iterate to a new schema.
  We already have multiple iterations of schemas.

  In the first iteration, we store all the columns needed to compute the interaction score, hashtag score and keyword score. The schema is as below:

```
root
 |-- created_at: string (nullable = true)
 |-- hashtag_list: array (nullable = true)
 |    |-- element: string (containsNull = true)
 |-- id: long (nullable = true)
 |-- in_reply_to_user_id: long (nullable = true)
 |-- is_reply: boolean (nullable = true)
 |-- is_retweet: boolean (nullable = true)
 |-- retweeted_status_user_id: long (nullable = true)
 |-- text: string (nullable = true)
 |-- user_description: string (nullable = true)
 |-- user_id: long (nullable = true)
 |-- user_screen_name: string (nullable = true)
```

  In the second iteration, we changed hashtags from Array<String> representation into String representation, which will be easier to store in MySQL database and for java to

process the query result.

```
mysql> describe tweets;
+---------------------------+--------+------+-----+---------+-------+
| Field                     | Type   | Null | Key | Default | Extra |
+---------------------------+--------+------+-----+---------+-------+
| id                        | bigint | YES  | MUL | NULL    |       |
| text                      | text   | YES  |     | NULL    |       |
| is_reply                  | bit(1) | NO   |     | NULL    |       |
| is_retweet                | bit(1) | NO   |     | NULL    |       |
| user_id                   | bigint | YES  | MUL | NULL    |       |
| user_description          | text   | YES  |     | NULL    |       |
| user_screen_name          | text   | YES  |     | NULL    |       |
| in_reply_to_user_id       | bigint | YES  | MUL | NULL    |       |
| retweeted_status_user_id  | bigint | YES  | MUL | NULL    |       |
| created_at                | text   | YES  |     | NULL    |       |
| hashtags                  | text   | NO   |     | NULL    |       |
+---------------------------+--------+------+-----+---------+-------+
11 rows in set (0.00 sec)
```

In the third iteration, which is our current last iteration. We have 2 more tables, which are the interaction score table and hashtag score table. We pre-compute the interaction score and hashtag score during the ETL process. So instead of getting data and then computing the scores in the web-tier, we just directly query the two tables to get the corresponding score in the web-tier. This definitely saves a lot of time. The schema of these 2 score tables are as below:

```
mysql> describe interaction_score;
+-------------------+--------+------+-----+---------+-------+
| Field             | Type   | Null | Key | Default | Extra |
+-------------------+--------+------+-----+---------+-------+
| user1             | bigint | NO   | MUL | NULL    |       |
| user2             | bigint | NO   |     | NULL    |       |
| interaction_score | double | NO   |     | NULL    |       |
+-------------------+--------+------+-----+---------+-------+
3 rows in set (0.00 sec)
```

```
mysql> describe hashtag_score;
+---------------+--------+------+-----+---------+-------+
| Field         | Type   | Null | Key | Default | Extra |
+---------------+--------+------+-----+---------+-------+
| user_id_a     | bigint | YES  |     | NULL    |       |
| user_id_b     | bigint | YES  |     | NULL    |       |
| commonTags    | int    | NO   |     | NULL    |       |
| hashtag_score | double | YES  |     | NULL    |       |
+---------------+--------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

- List 2 potential optimization that can be applied to your schemas. How might they improve your performance?

1. Combine interaction and hashtag data to reduce database round trips; omit contacts whose interaction score is zero.

2. Store all serving fields in a per-user JSON record. The contact_tweets query path used by the Java and C# services retrieves contact_tweet by user_id, then performs scoring in memory. Text and hashtag values use a shared separator, avoiding repeated queries for individual contacts.

Question 9: Optimizations and Tweaks

Frameworks and databases do not come in the best shape out-of-the-box. Each of them has tens of parameters to tune. Once you are satisfied with the web framework, you can start learning about the configuration parameters to see which ones might be the most relevant and gather data and evidence with each individual optimization. Although you might not have arrived at an optimized database schema yet, feel free to learn about tunable parameters in your database system. No need to feel obliged to test all of them now. In future phases, you will continue to try to improve the performance of your microservices.

Hint for Phase 2: A good schema will easily double or even triple the target RPS of your Microservice 3 without any parameter tuning. It is advised to first focus on improving your schema and get an okay performance with your best cluster configuration. If you are 10 times (an order of magnitude) away from the target, then it is very unlikely to make up the difference with parameter tunings.

- List as many possible items to tweak as you can find that might impact performance, in terms of: Kubernetes config, web framework, your code, DBs, DB connectors, OS, etc. Choose at least three optimizations, apply them one at a time and show a table with the microservice you are optimizing, the optimization you have applied and the RPS before and after applying it.
    1. In m3, we pre-compute the interaction score and the hashtag score during the ETL process and store the result to the database.
    2. In terms of Kubernetes config, within the budget, add more worker nodes and upgrade all the nodes.
    3. Use jobby and vert.x.
    4. Add indexes to MySQL database tables
    5. Add cache to m1 and m2.

	Before	After
4	Get stuck	53
2	53	98
1	98	133

- For every tweak you listed above, try to explain how it contributes to performance.

1. This will save lots of time in the web tier compared to computing the score in the web tier.
2. Using more worker nodes and upgrading nodes make the cluster handle more requests per second.
3. We compared lots of frameworks such as spring boot, jobby, vert.x, django, Flask. Jobby and vert.x are outstanding with higher performance.
4. This can significantly reduce the query time.
5. Because all datas is static, we can use a cache(hashMap) to improve performance and do not need to worry about stale data and cache invalidation.

- Which optimizations only exist in one type of DB and why? How can you simulate that optimization in the other (or if you cannot, why not)?
  Add indexes to MySQL database tables only exist in MySQL because it uses a B+ tree to store the data in the database. So instead of looking up the whole table, indexes can help to find the queried data quickly. We can use the DESCRIBE command in mysql console before and after adding indexes. If the index is correctly set, the number of rows needed to be looked up will significantly decrease. For HBase, we can optimize the row key design to speed up the query.

Question 10: Storage-tier deployment orchestration

In addition to your web-tier deployment orchestration, it is equally important to have automation and orchestration for your storage-tier deployment in order to save labor time and avoid potential human errors during your deployment.

- We require you to use Terraform, Kubernetes and Helm to orchestrate the deployment of your storage-tier. What steps were taken every time you deployed your system? If you use MySQL as your storage-tier, include your Helm chart and Kubernetes manifest in your code submission on GitHub. If you use HBase, include your Terraform script which provisions EMR in your submission. Starting with a Kubernetes cluster that has your web-tier running, we should be able to have a functional storage-tier that is reachable from your web-tier after running your deployment scripts.
  We choose to use MySQL as the storage tier. The scripts are similar to P2:

```
helm repo add stable https://charts.helm.sh/stable

helm repo update

export mysqlPassword=REDACTED

helm install mysql-m3 --set mysqlRootPassword=${mysqlPassword}

--set persistence.storageClass=gp2 --set persistence.size=60Gi

stable/mysql

kubectl port-forward svc/mysql-m3 3306 &
```

  After running these scripts, we installed a separate MySQL pod inside our Kubernetes cluster. The MySQL data will be stored in the newly created EBS volume, which is 60GB with type gp2. To access the MySQL server within the cluster, we can use DNS name

"mysql-m3.default.svc.cluster.local", user name "root", and password "REDACTED" (which is set to the environment variable). To access the MySQL server outside, we need to use a new service to expose the MySQL pod like this:

```
kubectl expose deployment mysql-m3 --type=LoadBalancer
--name=mysql-service
```

Then we can use the DNS of this load balancer service as the MySQL hostname to reach the database.

- Making snapshots of your database can save time importing your ETL results into the database if you need to re-deploy your storage-tier. What are some methods to make snapshots of your database? Did you implement any of them? If so, how much time have you saved each time you restore the snapshot, compared to having to import ETL results again?
  One method is to dump the database into a sql file using `mysqldump`. The other one is to make a snapshot of the EBS volume and later restore the volume from the snapshot and attach it to the MySQL pod. We tried the first method. Then we are able to run the sql file every time we start the cluster, instead of importing ETL results again, which saves us several hours.

Task 4: Site reliability

Question 11: Monitoring

Once the service is set up and running, it begins to generate a large amount of status data. They help us monitor if the system is operational, and also give us insights into its performance.

- What CloudWatch metrics are useful for monitoring the health of your system? What metrics help you understand performance?
  - Average Latency — Latency of load balancer.
  - Surge Queue Length — SurgeQueueLength, the total number of requests (HTTP listener) or connections (TCP listener) that are pending routing to a healthy instance.
  - HTTPCode_Backend_4XX of EC2 instance, which is our K8s node.
- What statistics can you collect if you have SSH access into the VM? What tools did you use? What did they tell you?
  - To view CPU utilization, *htop* would be helpful.
  - To check network and port connectivity, *netstat* will be helpful.
- What statistics can you collect if you have kubectl access, but not SSH access into the nodes? What tools did you use? What did they tell you?
  - kubectl could check whether the pod and node is ready/healthy or not.
  - kubectl top can monitor CPU utilization of pods.
  - If we have more time, we could also try *Prometheus*, a CNCF project, can natively monitor Kubernetes, nodes.
- How do you monitor the status of your database? What interesting facts did you observe?

    - First monitor availability, checking whether the database can be reached through command lines and JDBC
    - Second monitor resource consumption with CloudWatch, like disk I/O per second
    - We found database performance got slow under pressure such as expensive queries

Question 12: Profiling

We expect you to know the end-to-end latency of your system. This helps us to focus on optimizing the parts of the system that contribute to most of the latency.

- Given a typical request-response for Microservice 3, for each latency below, think about if you can measure it, how you can measure it, and write down the tools/techniques that can be used to measure it:
    - Load Generator to Load Balancer
        - This latency is caused by network latency. The HTTP packet travels from load generator and be transmitted to load balancer. We could use ping and traceroute to measure the time elapses between the moment the request is sent and the moment it reaches the load balancer IP address.
    - Load Balancer to Worker Node
        - Since we deploy the load balancer in AWS, then *CloudWatch* could be used to visualize the latency between load generator and load balancer. The metrics we might be interested in are **SurgeQueueLength** metric of the ELB. Specifically, sum statistic of the SpilloverCount, which measures the number of rejected request due to long queue and **Latency**, which
    - Worker Node to Web-tier Pod
        - Kubernetes provides metrics on the number and duration of requests to the API server for each combination of resource (e.g., pods, Deployments) and verb (e.g., GET, LIST, POST, DELETE). The end-to-end latency metrics report both how long it takes to select a node for a particular pod, as well as how long it takes to notify the API server of the scheduling decision.
    - Parsing request
        - We could use *VisualVM* to observe the time consumed by parsing requests in Java applications.
    - Web Service to DB
        - We could add *stopwatches* in our application code to observe the round trip time between DB and web service plus the SQL execution time. The single trip time is roughly equal to (the total time - execution time) / 2
    - At DB (execution)
        - we could manually test the database execution time by issuing a query. Then, we could use *"EXPLAIN select * from table"* to check if there are any indexes that could help accelerate the execution.
    - DB to Web Service
        - Similar to "Web Service to DB".
    - Parsing DB response

        - We could add stopwatches in our application code to observe how much time was spent to parse each DB response.
    - Web-tier Pod to Worker Node
        - Kubernetes provides metrics like `apiserver_request_latencies_sum` to track latencies of a specific request, including the latency from web-tier pod to worker node. We can use this together with other metrics.
    - Worker Node to Load Balancer
        - Similar to the "Load Balancer to Worker Node" section, which can be measured by *CloudWatch*.
    - Load Balancer to Load Generator
        - Similar to "Load Generator to Load Balancer", which should be about the same network latency.
- For each part above, think about whether you can improve the latency or not, and list possible ways to reduce the latency.
    - We could improve the following latencies:
        - **DB execution time** by creating indexes and more suitable table schema.
        - **Web Service to DB** by 1) maintaining a JDBC connection pool. 2) add a cache mechanism such as storing results of history queries.
        - **Load Balancer to Worker Node** by configuring an appropriate load balancing type, network LB or Application LB.
        - **Worker Node to Web-tier Pod** by keeping the workload as balanced as possible across multiple pods in a node.

Task 5: General questions

Question 13: Scalability

In this phase, we serve read-only requests from tens of GBs of processed data. Remember this is just an infinitesimal slice of all the user-generated content on Twitter. Can your servers provide the same quality of service when the amount of data grows? What if we realistically allow updates to tweets? These are good questions to discuss after successfully finishing this phase. It is okay if you answer no, but we do want you to think about them and share your understanding.

- Would your design work as well if the quantity of data was 10 times larger? What about 100x? Why or why not? (Assume you get a proportional amount of machines.)
    - With the appropriate amount of machines, I think our ETL design work could work because the ETL logic remains the same.
- Would your design work if your web service also implemented insert/update (PUT) requests? Why or why not?
    - I think we might encounter some problems if insert/update is involved. Because we have some pre-computed tables which use historical data. Suppose the data becomes real-time, then these tables are no longer valid and we cannot afford to compute these tables all over again just for one update/insert.

Question 14: Postmortem

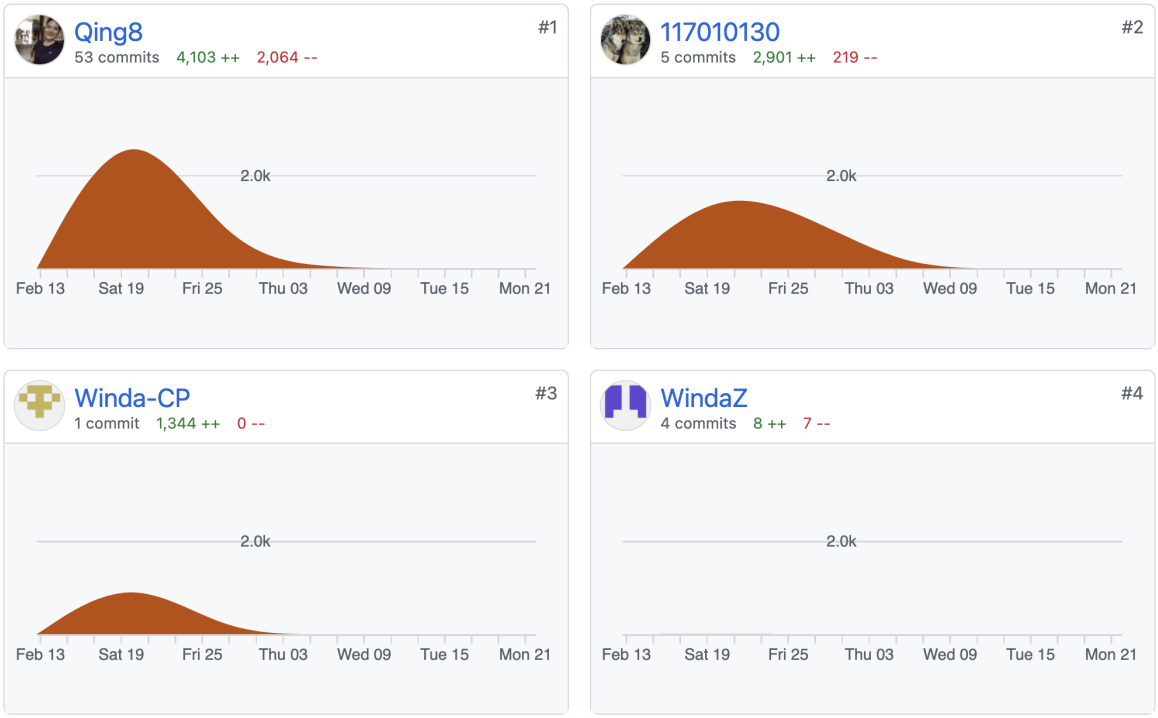
- How did you set up your local development environment? Did you install the databases locally on your machines to experiment? Did you test all programs before running them on your Kubernetes cluster on the cloud?
    - We set up our local development environment and install the database service on an EC2 instance. All programs are tested before running on K8s cluster.
- Did you attempt to generate a load to test your system on your own? If so, how? And why?
    - Yes. We need to profile the Java application and resolve the bottleneck, which will use JMeter to generate loads.
- Describe an alternative design to your system that you wish you had time or budget to try.
    - We could have also tried to utilize NoSQL databases such as MongoDB to store tweets.
- Which were the toughest roadblocks that your team faced in Phase 1?
    - One of the roadblocks is that we don't know how to utilize spark to its full potential. The ETL is so slow and is the most time consuming part of the project.
- Did you do something unique (any cool optimization/trick/hack) that you would like to share?
    - We did some configuration in *my.cnf* of MySQL database to help accelerate loading data into MySQL database. Otherwise, it might take days to load the backup sql files into DB.

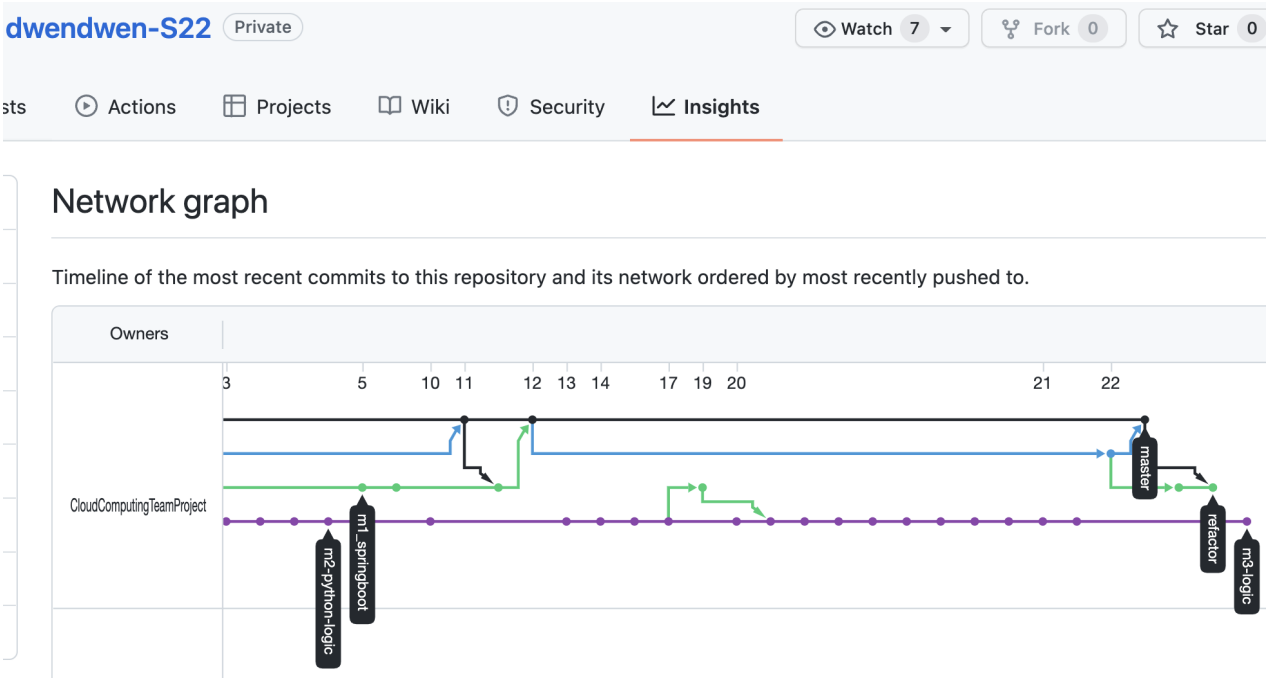
Question 15: Non-technical

- How did you divide components in the system? How do the components interact? How did you partition the work and what was each team member's contribution?
  We divided the system into ETL, Java backend, and deployment. For ETL, we further divided it into data filtering, interaction score calculation, hashtag score calculation, data loading. The deployment part will deploy all services and the MySQL database. Other parts can access the databases through its external IP. For example, the ETL part will load the data into the database, and the Java backend will retrieve the data from the database. In our team, Wangshu mainly focused on the deployment and the framework of the Java backend. Qing mainly focused on data filtering, optimizing Spark Scala code, designing database schema and loading data into MySQL database. Jiarui mainly focused on calculating interaction scores and hashtag scores and loading them into MySQL databases.
- How did you test each component? Did you implement Unit Testing or Integration Testing?
  We test the correctness using small datasets. In the Java backend, we implemented Unit Testing to test the output compared with the output from the reference server.
- Did you like the Git workflow, code review, and internal testing requirements? Were they helpful to your development?

Yes, they were helpful, because we could create different branches to work together and were able to review others' codes.

- Please show Github stats to reflect the contribution of each team member. Specifically, include screenshots for each of the links below.
  1. https://github.com/CloudComputingTeamProject/<repo>/graphs/contributors
  2. https://github.com/CloudComputingTeamProject/<repo>/network





Task 6: Bonus

5% Microservice 2 Early Bird Bonus

Fill in the form if your team was able to achieve the Microservice 2 target by 03/06 and hence eligible for the 5% early bird bonus. Please push all the files you wrote to reach the target RPS and tag your git commit as “earlyBird-m2” so that we know you completed this bonus at that commit.

	M2
score	50.00
submission id	696454
throughput	41001.58
latency	9.65
correctness	88.35
error	0.00
date/time	2022-03-04 22:15:49

Penalty Waiver for Microservice 2 Correctness

Fill in the form if your team was able to make a 10-minute Microservice 2 submission with above 95% correctness by 03/06 and hence eligible for waiving one most significant penalty for each team member.

	M2
score	
submission id	
throughput	
latency	
correctness	
error	
date/time	

5% Microservice 3 Early Bird Bonus

Fill in the form if your team was able to achieve the Microservice 3 phase 1 target (2000 RPS) by 03/20 and hence eligible for the 5% early bird bonus. Please push all the files you wrote to reach the target RPS and tag your git commit as “earlyBird-m3” so that we know you completed this bonus at that commit.

	M2
score	
submission id	
throughput	
latency	
correctness	
error	
date/time	

Penalty Waiver for Microservice 3 Correctness

Fill in the form if your team was able to make a 10-minute Microservice 3 submission with above 95% correctness by 03/20 and hence eligible for waiving one most significant penalty for each team member.

	M2
score	
submission id	
throughput	
latency	
correctness	
error	
date/time	